



DOM

Aula 10 · Bloco 3 — O Navegador e o DOM

JavaScript controlando a página

Nesta aula

1. O que é o **DOM**
2. **Selecionar** elementos
3. Alterar **conteúdo**, **estilo** e **classes**
4. **Criar** e **remover** elementos
5. Renderizar um **array** na tela

O que é o DOM

Quando o navegador carrega um HTML, ele o transforma numa **estrutura de objetos na memória**: o **DOM** — *Document Object Model*.

Cada tag vira um objeto que o JavaScript pode **ler e modificar**. E a página se atualiza na hora.

A porta de entrada é o objeto global `document`.

Selecionar: dois métodos resolvem tudo

Ambos usam a **sintaxe de seletores do CSS** — a mesma da aula passada.

```
// querySelector: O PRIMEIRO que casa com o seletor  
const titulo = document.querySelector("#titulo");    // por id  
const aviso  = document.querySelector(".aviso");    // por classe – só o 1º!  
const lista  = document.querySelector("ul");        // por tag  
  
// querySelectorAll: TODOS (uma NodeList, que aceita forEach)  
const avisos = document.querySelectorAll(".aviso");  
avisos.forEach((el) => console.log(el.textContent));
```

⚠ O erro nº 1 deste bloco

Se o seletor não acha nada, `querySelector` devolve `null` —

e a linha seguinte explode:

```
Cannot read properties of null
```

Duas causas, nessa ordem:

1. o seletor está errado;
2. o `<script>` não está no fim do `<body>`.

Alterando conteúdo

```
const titulo = document.querySelector("#titulo");

titulo.textContent = "Painel Hackeado 😎";           // troca o TEXTO
titulo.innerHTML = "Painel <em>Hackeado</em>";       // interpreta HTML
```

✏️ Prefira **textContent**. Use **innerHTML** só quando precisar inserir tags — e **nunca** com conteúdo digitado pelo usuário: é uma porta aberta para injeção de código.

Estilos e classes

```
titulo.style.color = "white";  
titulo.style.backgroundColor = "darkblue";    // background-color → camelCase  
  
titulo.classList.add("destaque");  
titulo.classList.remove("destaque");  
titulo.classList.toggle("destaque");        // liga/desliga  
titulo.classList.contains("destaque");      // true/false
```

💡 **Padrão do mercado:** o **CSS** define as aparências possíveis (`.destaque` , `.erro`); o **JS** só decide **quando** aplicá-las. Separação de responsabilidades.

Criar e remover: criar → configurar → inserir

```
const lista = document.querySelector("#lista-frutas");

const novoItem = document.createElement("li");    // criar
novoItem.textContent = "Uva";                      // configurar
novoItem.classList.add("fruta-nova");
lista.appendChild(novoItem);                        // inserir

document.querySelector("#lista-frutas li").remove(); // remover
```

Decore a sequência **criar → configurar → inserir**: ela se repete o bloco inteiro.

O padrão mais importante do bloco

Seus **dados** virando **página**.

```
produtos.forEach((p) => {  
  const li = document.createElement("li");  
  li.textContent = `${p.nome} - R$ ${p.preco}`;  
  lista.appendChild(li);  
});
```

Um array de objetos entra. Uma lista na tela sai.

E tudo do Bloco 2 se conecta aqui

Quer mostrar **só os baratos**?

```
produtos.filter((p) => p.preco < 100).forEach(...)
```

Quer **ordenados**? `.sort(...)` antes.

💡 **DOM = os seus dados + os métodos que você já domina.** Não há técnica nova aqui — é `map`, `filter` e `forEach` desembocando numa tela.

Exercícios da aula

Na pasta `aula-10/` :

1. **Cartão de perfil** — trocar nome e bio pelo JS, pintar uma div;
2. **Semáforo de classes** — `.ok` , `.atencao` , `.erro` via `classList.add` ;
3. **Lista gerada** — array de linguagens → `<li>` com `createElement` ;
4. **Tabela de alunos** — o array da aula 08 renderizado, com classe por situação;
5. **Desafio** 🌶️ — duas listas (todos / só aprovados) com uma função `renderizarLista` reaproveitada.

Próxima aula

Aula 11 — Eventos

A página já muda. Falta ela **reagir**
ao que a pessoa faz.